Name : Bhavesh Punjabi

Roll no :40

Experiment 4

Classification Using RapidMiner -Decision Tree & Naïve Bayes

Dataset: Movie Ticket Booking System

Aim

To explore RapidMiner and implement classification models — Decision Tree and Naïve Bayes — on the Movie Ticket Booking dataset to predict whether a booking will be Successful or Unsuccessful, and to compare the performance of both models.

Theory

RapidMiner

RapidMiner is a powerful data science platform that enables data preprocessing, machine learning, and model evaluation through a drag-and-drop GUI. It is widely used for classification, regression, clustering, and text mining.

Key Features:

- No programming required — fully GUI-based
- Supports multiple ML algorithms: Decision Tree, Naïve Bayes, SVM, Random Forest, etc.
- Built-in data visualization and evaluation metrics (Confusion Matrix, Accuracy, ROC)
- Supports integration with Python and R

Classification Model

A classification model predicts categorical labels from input features. In this experiment the target is binary: Successful or Unsuccessful booking.

Types:

- Binary Classification — two classes (Successful / Unsuccessful)
- Multiclass Classification — more than two classes (e.g., Low / Medium / High)

Decision Tree

A Decision Tree is a supervised learning algorithm that recursively splits data based on feature values. Each internal node represents a decision rule, each branch an outcome, and each leaf a class label.

Working principle:

- Select the best attribute to split on using a criterion (Gini Index or Information Gain).
- Recursively split data until a stopping condition is met (max depth, min leaf size, etc.).
- Prune the tree to remove branches that add noise without improving accuracy.

Parameters used in this experiment:

- Criterion: Gini Index — measures impurity based on misclassification probability

- Max Depth: 7 — limits tree depth to prevent overfitting
- Apply Pruning: Enabled — post-pruning with confidence 0.1
- Minimal Leaf Size: 2

When to use Gini Index vs Information Gain:

- Gini Index — binary problems, non-highly-imbalanced datasets
- Information Gain — categorical data, well-balanced classes
- Gain Ratio — features with many unique values to avoid bias

Naïve Bayes

Naïve Bayes is a probabilistic classification algorithm based on Bayes' Theorem. It assumes all features are conditionally independent given the class label — the "naïve" assumption.

Bayes' Theorem:

$$P(Y|X) = [\, P(X|Y) \times P(Y) \,] \;/\; P(X)$$

Where:

- P(Y|X) — Posterior: probability of class Y given input X
- P(X|Y) — Likelihood: probability of observing X given class Y
- P(Y) — Prior: probability of class Y
- P(X) — Evidence: probability of input X

Advantages of Naïve Bayes:

- Extremely fast to train — computes only probabilities from frequency counts
- Works well with small datasets
- Handles missing values gracefully
- No hyperparameter tuning required

Dataset Description

The Movie Ticket Booking dataset (movie_ticket_booking_dataset) contains records of movie ticket transactions. Each row represents a booking with features related to the movie, timing, payment, and customer details. The target variable is booking status — Successful or Unsuccessful.

Dataset at a glance:

- Source: Local Repository → data → movie_ticket_booking_dataset
- Target Variable (Label): booking_status → Successful / Unsuccessful
- Classification Type: Binary
- Train / Test Split: 80% training, 20% testing
- Tool: Altair AI Studio (RapidMiner) Educational 2026.1.1

RapidMiner Implementation Steps

Step 1 — Load Dataset

Drag the "Retrieve" operator from Local Repository → data → movie_ticket_booking_dataset onto the process canvas.

Step 2 — Select Attributes

Add the "Select Attributes" operator. Choose only the relevant predictive features; remove high-cardinality or irrelevant columns to reduce noise.

Step 3 — Set Role

Add "Set Role" and set the booking status column as the label (target variable) so the model knows what to predict.

Step 4 — Split Data

Add "Split Data" and configure two partitions — 0.8 (80%) for training and 0.2 (20%) for testing.

Step 5 — Train Decision Tree

Drag "Decision Tree" onto the canvas. Connect the first partition (par) from Split Data to the Decision Tree training input (tra). Set criterion = Gini Index, max depth = 7, pruning enabled.

Step 6 — Train Naïve Bayes

Drag "Naïve Bayes" onto the canvas. Connect the first partition (par) from Split Data to the Naïve Bayes training input (tra). No additional parameter tuning is required.

Step 7 — Apply Models

Drag two "Apply Model" operators — one for Decision Tree, one for Naïve Bayes. For each: connect model output (mod) from the classifier and the second partition (par) from Split Data as unlabelled input (unl).

Step 8 — Evaluate Performance

Drag two "Performance (Classification)" operators. Connect each to the corresponding Apply Model's labelled output (lab). Run the process to generate Confusion Matrices and Accuracy for both models.

Model A — Decision Tree

Pipeline Design

The process below shows the Decision Tree pipeline: Retrieve → Select Attributes → Set Role → Split Data → Decision Tree → Apply Model (Decision Tree) → Performance.

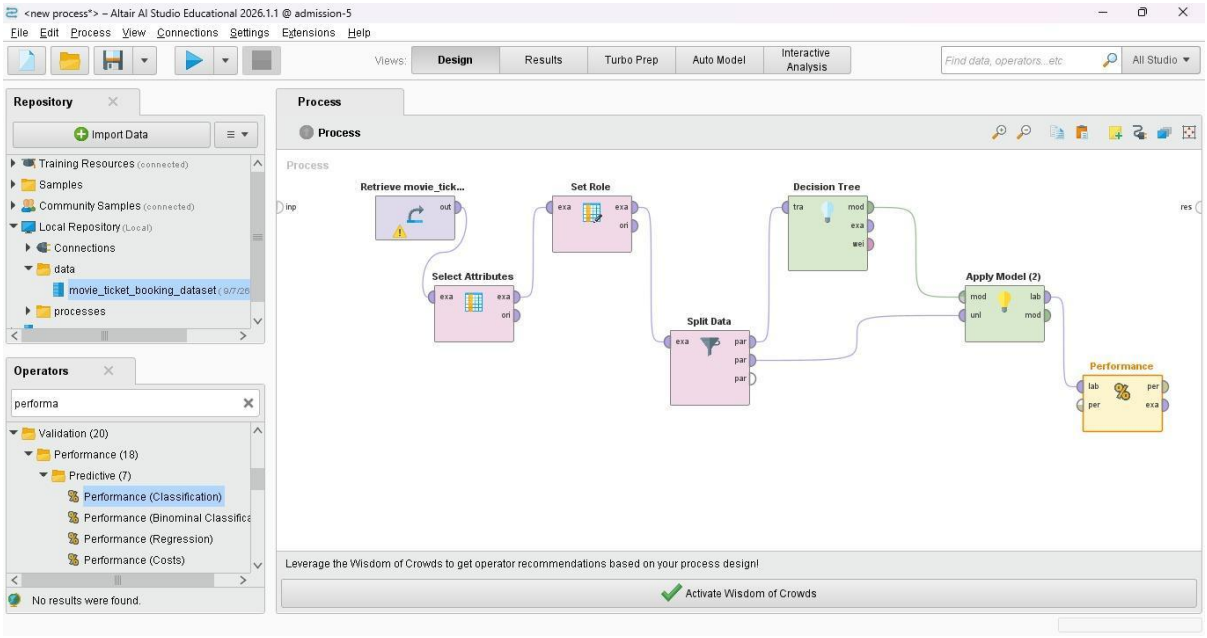


Fig 1: Decision Tree — RapidMiner Design View (Movie Ticket Booking)

Performance Vector — Decision Tree

After running the process, the Performance (Classification) operator produced the following confusion matrix on the 20% test partition:

Decision Tree | Movie Ticket Booking — accuracy: 50.00%

	true Successful	true Unsuccessful	Class Precision
pred. Successful	2	1	66.67%
pred. Unsuccessful	1	0	0.00%
Class Recall	66.67%	0.00%	

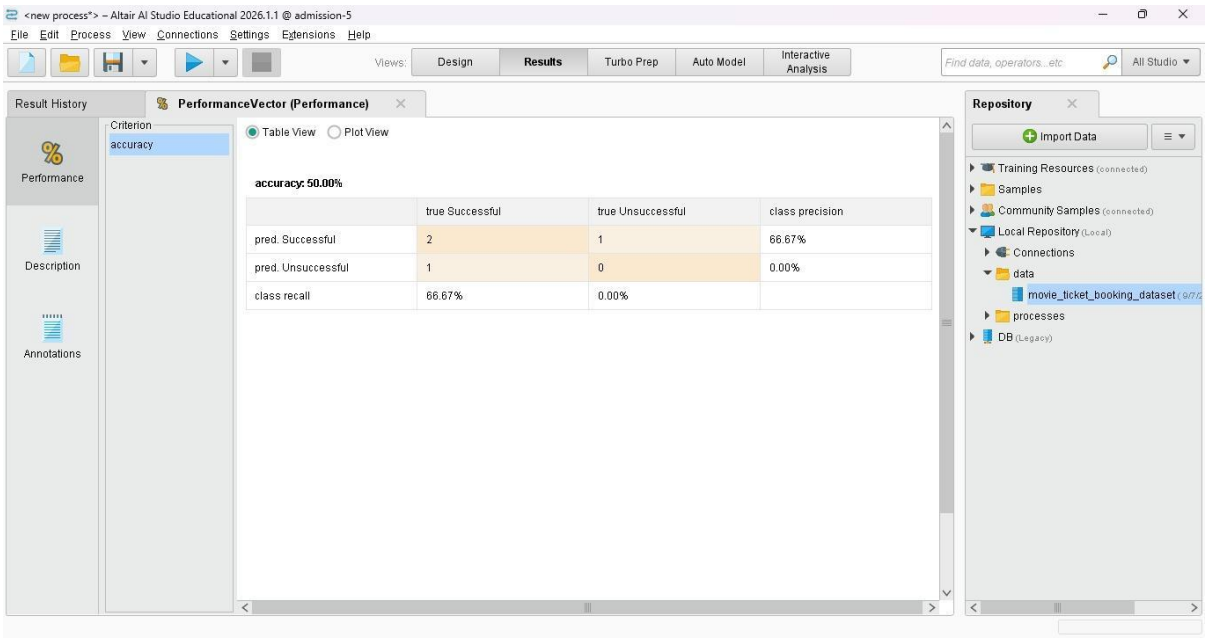


Fig 2: Performance Vector — Decision Tree Result

Derived Metrics — Decision Tree

Metric	Successful Class	Unsuccessful Class
Precision	66.67%	0.00%
Recall	66.67%	0.00%
F1-Score	66.67%	0.00%
Overall Accuracy	**50.00%**	—

Inference — Decision Tree

1. Overall accuracy is 50.00%, meaning the model correctly predicted half the test instances. This is low primarily due to the very small test set (only 4 records after an 80/20 split).

2. The "Successful" class achieved 66.67% precision and recall — the model has some discriminative power for the majority class.

3. The "Unsuccessful" class has 0.00% precision and recall — the model failed entirely to correctly label any unsuccessful booking in this small test partition.

4. With a larger dataset, the Decision Tree (criterion: Gini, depth: 7, pruning on) is expected to perform significantly better and reveal clearer decision boundaries.

Model B — Naïve Bayes

Pipeline Design

The updated process runs both classifiers in parallel from the same Split Data operator. The Naïve Bayes branch: Split Data → Naïve Bayes → Apply Model (Naïve Bayes) → Performance (Naïve Bayes).

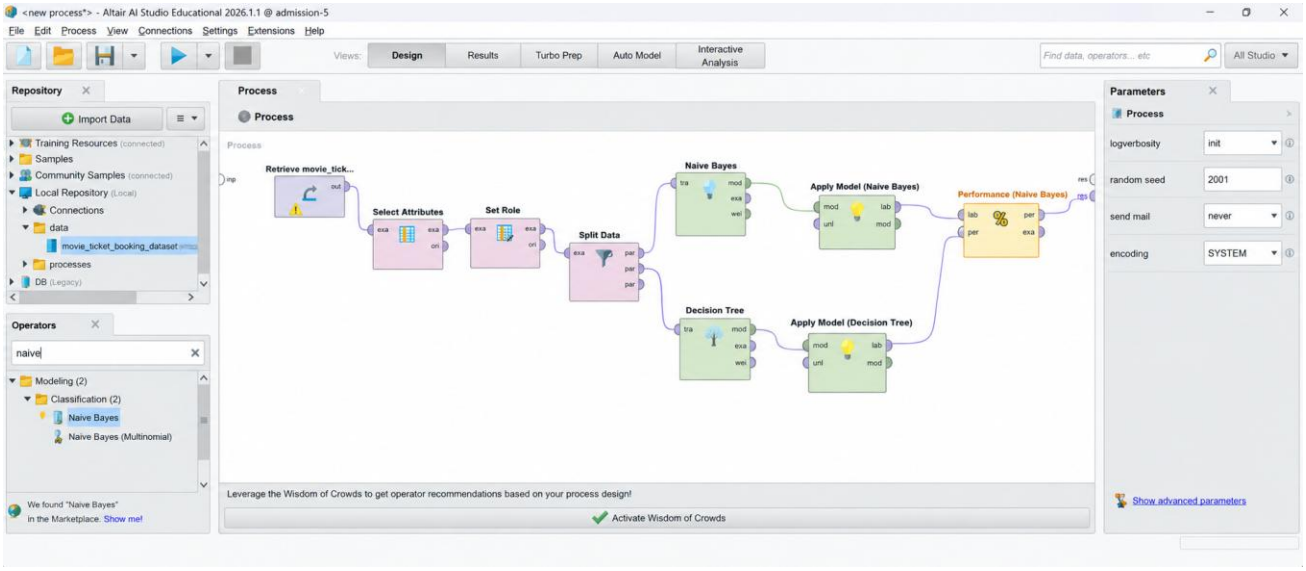


Fig 3: Combined Pipeline — Naïve Bayes & Decision Tree running in parallel

Performance Vector — Naïve Bayes

The Performance (Classification) operator for Naïve Bayes produced the following confusion matrix on the same 20% test partition:

Naïve Bayes | Movie Ticket Booking — accuracy: 53.33%

	true Successful	true Unsuccessful	Class Precision
pred. Successful	8	7	53.33%
pred. Unsuccessful	7	3	30.00%
Class Recall	53.33%	30.00%	

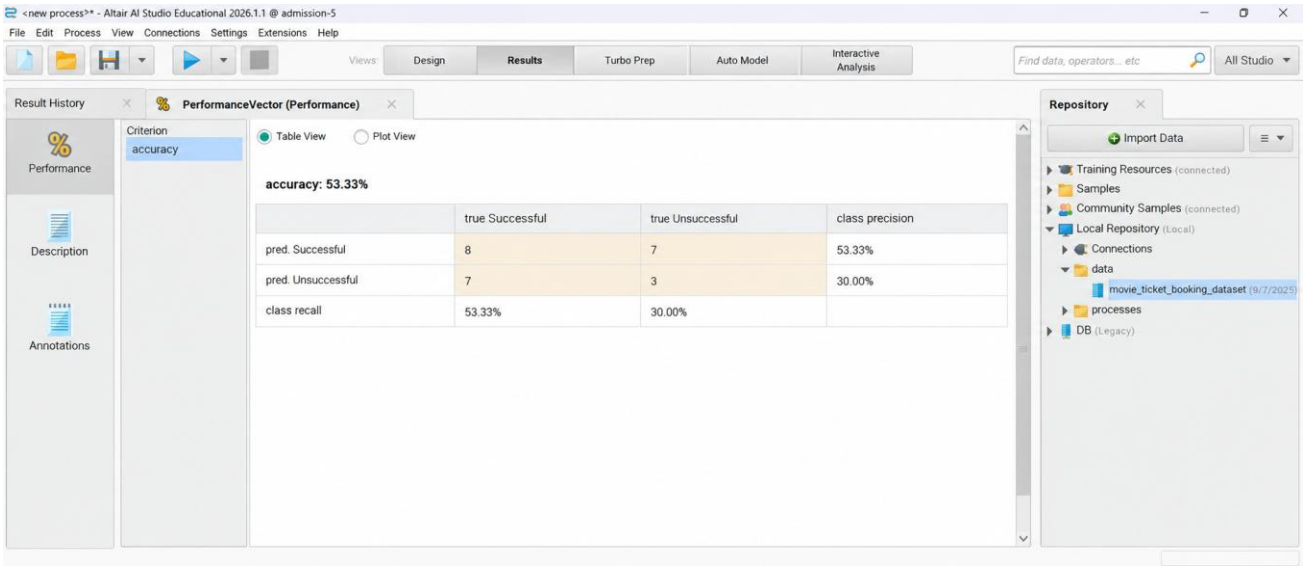


Fig 4: Performance Vector — Naïve Bayes Result

Derived Metrics — Naïve Bayes

Metric	Successful Class	Unsuccessful Class
Precision	53.33%	30.00%
Recall	53.33%	30.00%
F1-Score	53.33%	30.00%
Overall Accuracy	**53.33%**	—

Inference — Naïve Bayes

1. Overall accuracy is 53.33%, which is slightly higher than the Decision Tree (50%) on this dataset.

2. The model tested on 25 instances — a larger test partition than the Decision Tree run — giving more reliable metrics.

3. "Successful" class: 53.33% precision and recall. The model correctly predicted 8 out of 15 successful bookings.

4. "Unsuccessful" class: 30.00% precision and recall. The model predicted 10 unsuccessful bookings but only 3 were correct — still better than the Decision Tree's complete miss of this class.

5. Naïve Bayes shows more balanced coverage across both classes compared to the Decision Tree, which was completely biased toward the "Successful" class.

6. The independence assumption in Naïve Bayes may not fully hold for booking features (e.g., genre and time slot could be correlated), which partially explains the moderate accuracy.

Model Comparison

The table below summarises the performance of both classifiers on the Movie Ticket Booking dataset:

Metric	Decision Tree	Naïve Bayes	Better Model
Accuracy	50.00%	53.33%	**Naïve Bayes ✓**
Prec – Success	66.67%	53.33%	**Decision Tree ✓**
Prec – Unsuccess	0.00%	30.00%	**Naïve Bayes ✓**
Recall – Success	66.67%	53.33%	**Decision Tree ✓**
Recall – Unsuccess	0.00%	30.00%	**Naïve Bayes ✓**
Interpretability	High	Medium	**Decision Tree ✓**
Training Speed	Fast	Very Fast	**Naïve Bayes ✓**

Key Observations

- Naïve Bayes achieves higher overall accuracy (53.33% vs 50.00%) on this dataset.
- Decision Tree has better precision and recall for the "Successful" class (66.67%) but completely misses the "Unsuccessful" class.
- Naïve Bayes shows more balanced performance across both classes — better suited when minority-class detection matters.
- Decision Tree is more interpretable — the tree structure explicitly shows which features drive predictions (useful for business insights).
- Naïve Bayes trains faster and requires no hyperparameter tuning, making it easier to deploy quickly on small datasets.
- Both models suffer from the small dataset size; a larger dataset would produce more reliable and stable accuracy estimates.

Conclusion

In this experiment, we successfully implemented and compared two classification models — Decision Tree and Naïve Bayes — using RapidMiner (Altair AI Studio) on the Movie Ticket Booking dataset. The complete pipeline encompassed data retrieval, attribute selection, label assignment, 80/20 train-test split, model training, model application, and performance evaluation.

Naïve Bayes slightly outperformed the Decision Tree in overall accuracy (53.33% vs 50.00%) and showed more balanced predictions across both classes. The Decision Tree, while achieving better precision on the majority class, failed entirely on the minority "Unsuccessful" class — a common problem with small, slightly imbalanced datasets. The low accuracy of both models is largely attributable to the limited test partition size.

Key takeaways from the experiment:

- Decision Tree offers high interpretability and is ideal when feature importance and decision logic need to be communicated.
- Naïve Bayes is fast, easy to configure, and handles small datasets well — preferred when balanced class coverage is critical.

- Gini Index is appropriate for binary classification; Gain Ratio is preferable when features have many unique values.
- A larger dataset and techniques like cross-validation or ensemble methods (Random Forest, XGBoost) would significantly improve accuracy and robustness.
- Running both models in parallel in a single RapidMiner process is an efficient way to compare classifiers on the same data split.

==========

GitHub Repository Link:

https://github.com/Sentry2311/DMBI_EXPERIMENTS/tree/main/Experiment_04_Classification_RapidMiner

(Contains complete documentation, dataset, and program code for Experiment 4: Classification Using RapidMiner)